

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import Perceptron

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)
```

```
cancer = load_breast_cancer()

X = cancer.data

y = cancer.target
```

```
print("Dataset Description:\n")
print(cancer.DESCR)

print("\nDataset Shape:", X.shape)

print("Number of Samples:", X.shape[0])

print("Number of Features:", X.shape[1])

print("\nFeature Names:")
print(cancer.feature_names)

print("\nTarget Classes:")
print(cancer.target_names)
```

prognosis via linear programming. Operations Research, 43(4), pages 570-577, July-August 1995.

- W.H. Wolberg, W.N. Street, and O.L. Mangasarian. Machine learning techniques to diagnose breast cancer from fine-needle aspirates. Cancer Letters 77 (1994) 163-171.

Dataset Shape: (569, 30)
Number of Samples: 569
Number of Features: 30

Feature Names:
['mean radius' 'mean texture' 'mean perimeter' 'mean area'
'mean smoothness' 'mean compactness' 'mean concavity'
'mean concave points' 'mean symmetry' 'mean fractal dimension'
'radius error' 'texture error' 'perimeter error' 'area error'
'smoothness error' 'compactness error' 'concavity error'
'concave points error' 'symmetry error' 'fractal dimension error'
'worst radius' 'worst texture' 'worst perimeter' 'worst area'
'worst smoothness' 'worst compactness' 'worst concavity'
'worst concave points' 'worst symmetry' 'worst fractal dimension']

Target Classes:
['malignant' 'benign']

```
df = pd.DataFrame(X, columns=cancer.feature_names)

df["Class"] = y

df.head()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst texture	worst perimeter
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	17.33	184.60
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	23.41	158.80
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	25.53	152.50
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	26.50	98.87
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	16.67	152.20

5 rows × 31 columns

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   mean radius                           569 non-null    float64
1   mean texture                           569 non-null    float64
2   mean perimeter                         569 non-null    float64
3   mean area                             569 non-null    float64
4   mean smoothness                       569 non-null    float64
5   mean compactness                       569 non-null    float64
6   mean concavity                         569 non-null    float64
7   mean concave points                    569 non-null    float64
8   mean symmetry                          569 non-null    float64
9   mean fractal dimension                 569 non-null    float64
10  radius error                           569 non-null    float64
11  texture error                           569 non-null    float64
12  perimeter error                         569 non-null    float64
13  area error                             569 non-null    float64
14  smoothness error                       569 non-null    float64
15  compactness error                      569 non-null    float64
16  concavity error                         569 non-null    float64
17  concave points error                    569 non-null    float64
18  symmetry error                          569 non-null    float64
19  fractal dimension error                 569 non-null    float64
20  worst radius                           569 non-null    float64
21  worst texture                           569 non-null    float64
22  worst perimeter                         569 non-null    float64
23  worst area                             569 non-null    float64
24  worst smoothness                       569 non-null    float64
25  worst compactness                       569 non-null    float64
26  worst concavity                         569 non-null    float64
27  worst concave points                    569 non-null    float64
```

```
28 worst symmetry          569 non-null    float64
29 worst fractal dimension  569 non-null    float64
30 Class                    569 non-null    int64
dtypes: float64(30), int64(1)
memory usage: 137.9 KB
```

```
df.describe()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...
count	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	...
mean	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.088799	0.048919	0.181162	0.062798	...
std	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.079720	0.038803	0.027414	0.007060	...
min	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000	0.000000	0.106000	0.049960	...
25%	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.029560	0.020310	0.161900	0.057700	...
50%	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.061540	0.033500	0.179200	0.061540	...
75%	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.130700	0.074000	0.195700	0.066120	...
max	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.426800	0.201200	0.304000	0.097440	...

8 rows × 31 columns

```
print(df.isnull().sum())
```

```
mean radius          0
mean texture         0
mean perimeter       0
mean area            0
mean smoothness      0
mean compactness     0
mean concavity       0
mean concave points  0
mean symmetry        0
mean fractal dimension 0
radius error         0
texture error        0
perimeter error      0
area error           0
smoothness error     0
compactness error    0
concavity error      0
concave points error 0
symmetry error       0
fractal dimension error 0
worst radius         0
worst texture        0
worst perimeter      0
worst area           0
worst smoothness     0
worst compactness    0
worst concavity      0
worst concave points 0
worst symmetry       0
worst fractal dimension 0
Class                0
dtype: int64
```

```
print("Duplicate Rows:", df.duplicated().sum())
```

Duplicate Rows: 0

```
print(df["Class"].value_counts())

plt.figure(figsize=(6,4))

sns.countplot(x="Class", data=df)

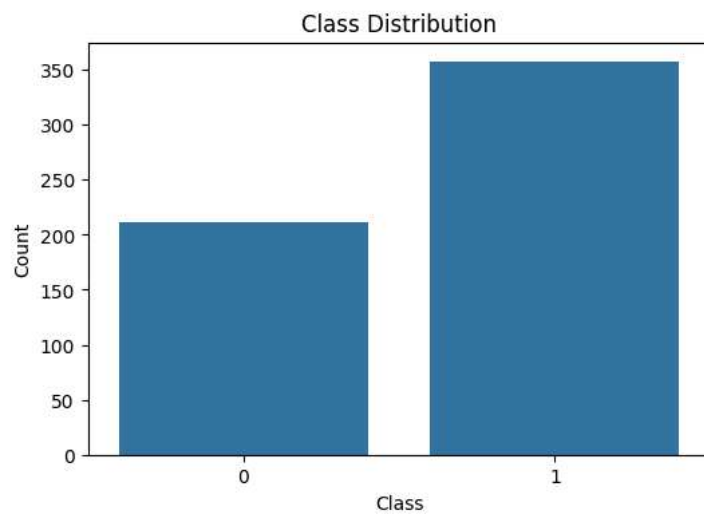
plt.title("Class Distribution")

plt.xlabel("Class")

plt.ylabel("Count")
```

```
plt.show()
```

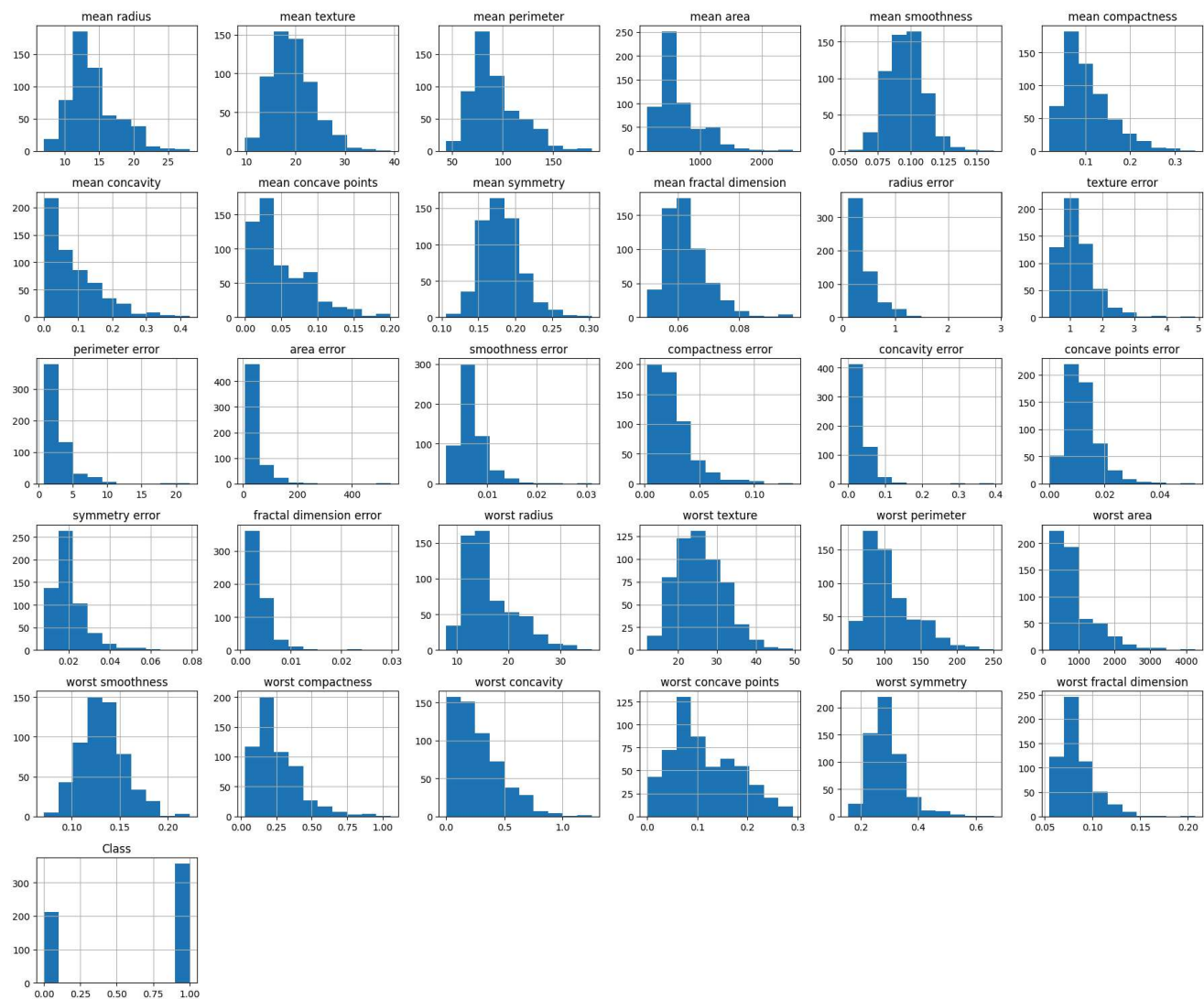
```
Class
1    357
0    212
Name: count, dtype: int64
```



```
df.hist(figsize=(18,15))
```

```
plt.tight_layout()
```

```
plt.show()
```



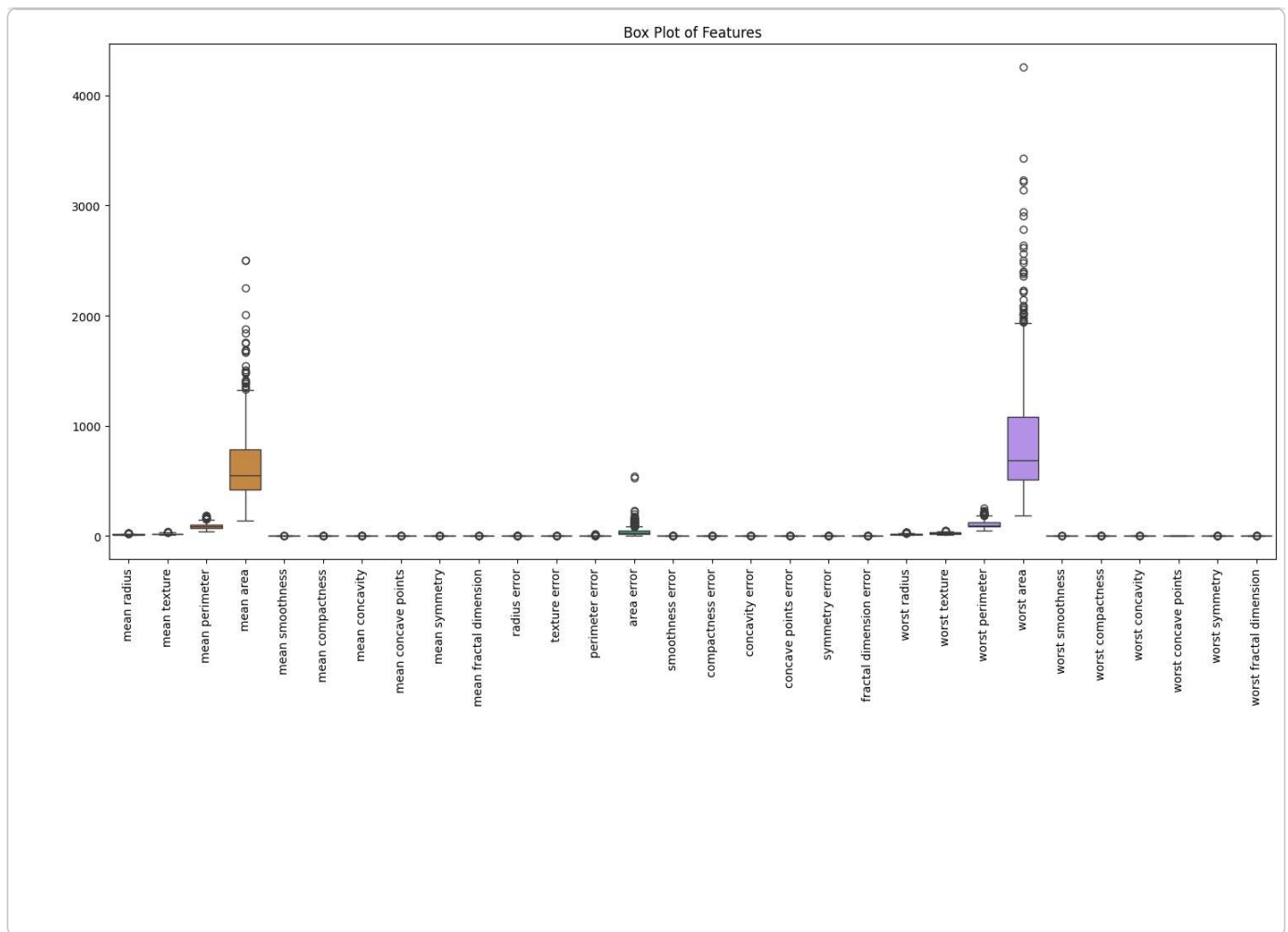
```
plt.figure(figsize=(18,8))

sns.boxplot(data=df.iloc[:, :-1])

plt.xticks(rotation=90)

plt.title("Box Plot of Features")

plt.show()
```

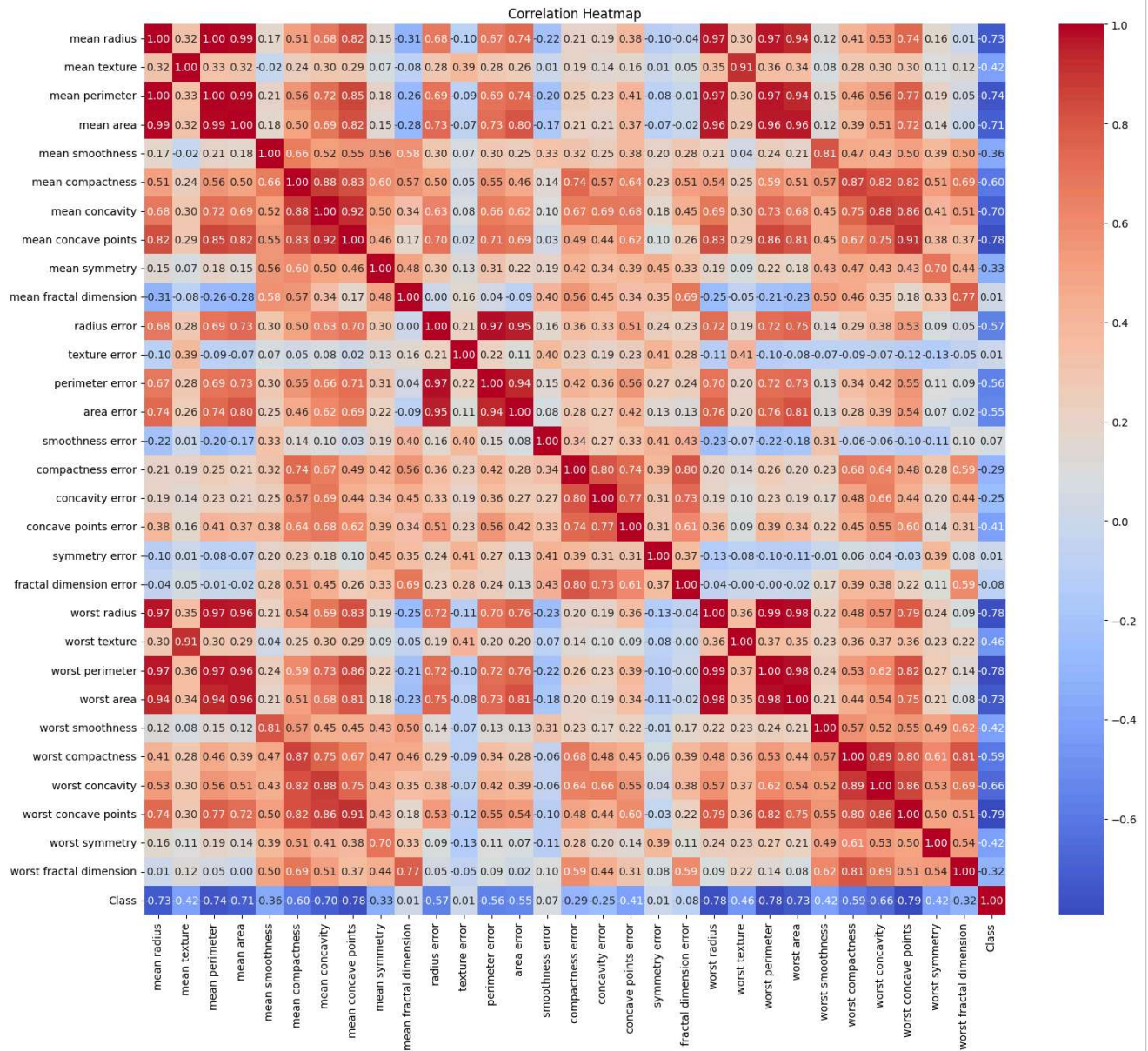


```
plt.figure(figsize=(18,15))

sns.heatmap(
    df.corr(),
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap")

plt.show()
```

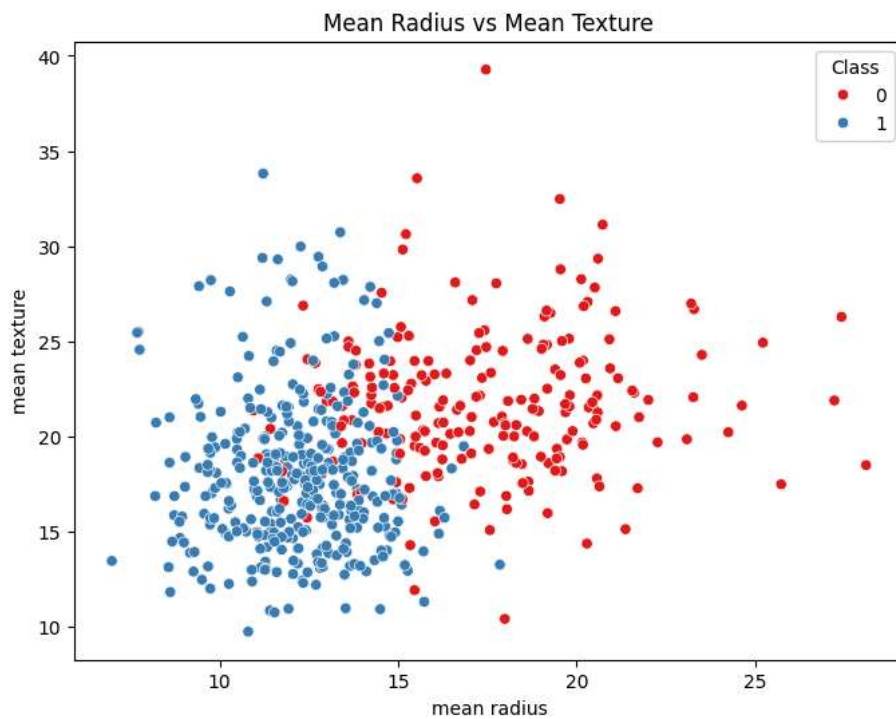


```
plt.figure(figsize=(8,6))

sns.scatterplot(
    data=df,
    x="mean radius",
    y="mean texture",
    hue="Class",
    palette="Set1"
)

plt.title("Mean Radius vs Mean Texture")
```

```
plt.show()
```



```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
```

```
model = Perceptron(
    max_iter=1000,
    eta0=0.01,
    random_state=42
)
```

```
model.fit(X_train, y_train)
```

```
▼ Perceptron ⓘ ?
Perceptron(eta0=0.01, random_state=42)
```

```
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", round(accuracy * 100, 2), "%")
```

```
Accuracy: 96.49 %
```

```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))

sns.heatmap(
```

```

cm,
annot=True,
fmt="d",
cmap="Blues",
xticklabels=cancer.target_names,
yticklabels=cancer.target_names
)

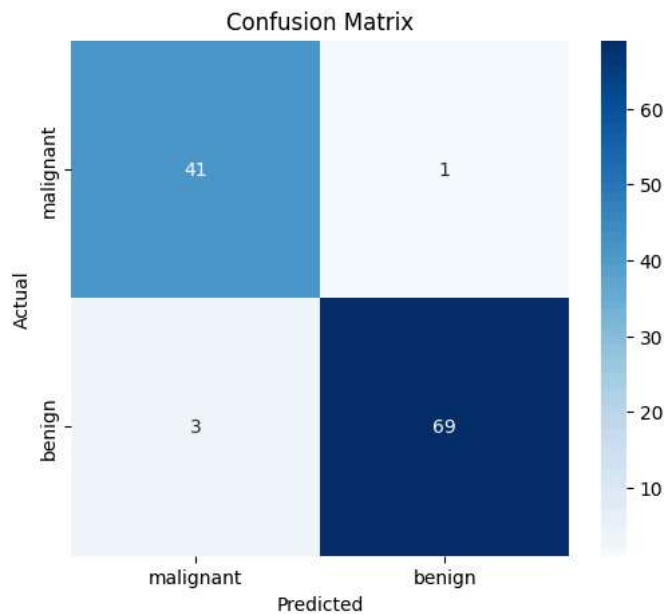
plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix")

plt.show()

```



```

print(classification_report(
    y_test,
    y_pred,
    target_names=cancer.target_names
))

```

	precision	recall	f1-score	support
malignant	0.93	0.98	0.95	42
benign	0.99	0.96	0.97	72
accuracy			0.96	114
macro avg	0.96	0.97	0.96	114
weighted avg	0.97	0.96	0.97	114

```

print("Weights of the Perceptron:\n")
print(model.coef_)

```

Weights of the Perceptron:

```

[[-0.04545231 -0.03970051 -0.04419493 -0.05015595 -0.03276819  0.06628719
 -0.01679867 -0.02907945  0.0367286  0.03750238 -0.08727599  0.03689474
 -0.07307851 -0.10143279 -0.08263657  0.00076058  0.08817379 -0.07313615
  0.01597395  0.0804158  -0.06430524 -0.1268126  -0.05746933 -0.07069104
 -0.04565509  0.00027175 -0.07701097 -0.05385808 -0.10541661 -0.01617949]]

```

```

print("Bias (Intercept):")
print(model.intercept_)

```

Bias (Intercept):
[-3.46944695e-18]

```
weights = model.coef_[0]
```

```

plt.figure(figsize=(12,6))

plt.bar(range(len(weights)), weights)

plt.xticks(range(len(weights)), cancer.feature_names, rotation=90)

plt.xlabel("Features")

plt.ylabel("Weight")

plt.title("Feature Weights Learned by the Perceptron")

plt.tight_layout()

plt.show()

```

